

과제1

Problem 1.

Write a Python function that takes a sequence of one or more numbers and returns the smallest and largest values as a tuple. Do not use the built-in functions min or max.

```
[1]:
def minmaxFinder (data ):
    if (data == None):
        return None,None

    ##최대, 최소값 구하는 로직.
    minVal = data[0]
    maxVal = data[0]

    for num in data:
        if num > maxVal:
            maxVal=num

        if num < minVal:
            minVal=num
    return minVal, maxVal

## 확인 예제1
list =[1,2,3,4,5,6,7]
minNum, maxNum = minmaxFinder(list)
print(f"min : {minNum}, max : {maxNum}")

## 확인 예제2
list =[11,5,6,7]
minNum, maxNum = minmaxFinder(list)
print(f"min : {minNum}, max : {maxNum}")

## 확인 예제3
list =[11,111,6,7]
minNum, maxNum = minmaxFinder(list)
print(f"min : {minNum}, max : {maxNum}")

min : 1, max : 7
min : 5, max : 11
min : 6, max : 111
```

Problem 2.

Given a string s of length n, if a negative index k ($-n \leq k < 0$) is used, determine the equivalent non-negative index j that refers to the same element.

Sol)

- 인덱스 -1은 문자열의 마지막 인덱스와 같고,

- 인덱스 -n은 문자열의 첫번째 인덱스와 같다.
- -n보다 크고 0보다 작은 k에 대응하는 인덱스는 뒤에서 -k번째 인덱스와 같다.
- 따라서, $n - (-k) = n + k$ 번째 인덱스와 같다.

아래 예제에서 이를 확인하겠다.

```
[2]: # 인덱스 -1은 문자열의 마지막 인덱스와 같고,
      # 인덱스 -n은 문자열의 첫번째 인덱스와 같다.

      # -n보다 크고 0보다 작은 k에 대응하는 인덱스는 뒤에서 -k번째 인덱스와 같다.
      # 따라서,  $n - (-k) = n + k$ 번째 인덱스와 같다.

      s="Structure"
      for i in range(len(s)-1, (-1)*len(s)-1,-1) :
          print(f"{i}번째 인덱스의 문자는 {s[i]}")
```

```
8번째 인덱스의 문자는 e
7번째 인덱스의 문자는 r
6번째 인덱스의 문자는 u
5번째 인덱스의 문자는 t
4번째 인덱스의 문자는 c
3번째 인덱스의 문자는 u
2번째 인덱스의 문자는 r
1번째 인덱스의 문자는 t
0번째 인덱스의 문자는 s
-1번째 인덱스의 문자는 e
-2번째 인덱스의 문자는 r
-3번째 인덱스의 문자는 u
-4번째 인덱스의 문자는 t
-5번째 인덱스의 문자는 c
-6번째 인덱스의 문자는 u
-7번째 인덱스의 문자는 r
-8번째 인덱스의 문자는 t
-9번째 인덱스의 문자는 s
```

Problem 3.

Algorithms (A) and (B) perform $8n \log n$ and $2n^2$ operations, respectively.
Find n_0 such that (A) is more efficient than (B) for all $n \geq n_0$

(A)가 (B)보다 성능이 좋은 시점은 $8n \log(n) < 2n^2$ 인 시점이다.

$4 \log(n) < n$ 인 시점

1).상용로그

- $n = 1$ 일때, $4 \log(n) = 0$, $n = 1$
- $n = 2$ 일때, $4 \log(n) = 4 \log(2)$, $n = 2$
- $n = 4$ 일때, $4 \log(n) = 4 \log(4)$, $n = 4$
- n_0 은 1일때 부터 (A)의 성능이 더 좋다.

2).로그의 밑이 2라면...

- $n = 1$ 일때, $4\log_2(n) = 0$, $n = 1$
 - $n = 2$ 일때, $4\log_2(n) = 4\log_2(2)$, $n = 2$
 - $n = 4$ 일때, $4\log_2(n) = 8\log_2(2)$, $n = 4$
 - $n = 16$ 일때, $4\log_2(n) = 16$, $n = 16$
 - $n = 17$ 일때, $4\log_2(n) = 4\log_2(17) = 16.35$, $n = 17$
 - 따라서, $n_0 = 17$ 이다.
-

Problem 4.

Order the following functions by their asymptotic growth rates:

$$4n \log n + 2n$$

$$2^{10}$$

$$2^{\log n}$$

$$3n + 100 \log n$$

$$4n$$

$$2^n$$

$$n^2 + 10n$$

$$n^3$$

$$n \log n$$

- 일반적으로 상수 < 로그 < 선형 < 선형 로그 < 다항식 < 지수의 순서를 가진다.
- 상수 시간 ($O(1)$) :

$$2^{10}$$

- 선형 시간($O(n)$) :

$$2^{\log n} < 3n + 100 \log n < 4n$$

- 선형 로그 시간($O(n \log n)$) :

$$n \log n < 4n \log n + 2n$$

- 다항식 시간($O(n^k)$) :

$$n^2 + 10n < n^3$$

- 지수 시간($O(2^n)$) :

$$2^n$$

Problem 5.

Give a Big-O characterization of the running time of the function example in Figure 1.

- 가장 안쪽 루프 반복 횟수 : 1~n번
 - 중간 루프 반복 횟수 : n번
 - 가장 바깥쪽 루프 반복 횟수 : n번
 - 따라서, $O(n^3)$ 이다.
-

Problem 6.

Write a recursive Python function to compute the n-th Harmonic number

- Harmonic Number :

$$H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}$$

```
[3]: def harmonicNum(n):
      if(n<=0) :
          return None
      if(n==1) :
          return 1
      return (1/n+harmonicNum(n-1))

# 동작 확인
for i in range(5,0,-1):
    print(f"n = {i:.4f}, sum = {harmonicNum(i):.4f}")

n = 5.0000, sum = 2.2833
n = 4.0000, sum = 2.0833
n = 3.0000, sum = 1.8333
n = 2.0000, sum = 1.5000
n = 1.0000, sum = 1.0000
```

Problem 7.

Write a recursive Python function that determines whether a string s is a palindrome or not, that is, it is equal to its reverse. For example, 'racecar' and 'kayak' are palindromes.

```
[4]: def palindromeFinder(s):
    if (len(s)==0 | len(s)==1):
        return True
    if (s[0] == s[-1]) :
        return palindromeFinder(s[1:-1])
    else :
        return False

strList = ["racecar","kayak", "hello", "madam", "computer"]

for str in strList:
    result = palindromeFinder(str)
    print(f"{str} : {result}")

racecar : True
kayak : True
hello : False
madam : True
computer : False
```

Problem 8.

Describe an algorithm for finding the second-to-last node in a singly linked list where the last node points to None.

- 단일 연결 리스트는 뒤로 못 가니까, 끝 직전까지 한 번만 순회해서 찾아야 한다
- 노드는 item, next 멤버 두 개로 이루어진다.
- 첫 시작 노드는 prev 라고 했을 때, prev.next는 다음 노드를 가리킨다.
- prev = prev.next를 통해서 prev는 2번째 노드를 가리킨다.
- 이 과정을 반복하고, prev.next.next가 None일때, prev는 마지막에서 2번째 노드를 가리킨다.
- 만약 노드가 0개 혹은 1개인 경우는 찾을 수가 없다.

Problem 9.

Describe an algorithm to swap two nodes x and y (not just their values) in a singly linked list. Repeat for a doubly linked list and compare which approach is more efficient. (Assume that references to x and y are provided.)

Singly Linked list

2개의 노드를 바꾸는 알고리즘은 아래와 같다.

1.head부터 시작해서 x의 이전 노드(prev1), y의 이전 노드(prev2)를 찾는다.

- (current.next=xNode 이면 prev1, current.next=yNode 이면 prev2)
- 2.prev1.next가 y를 가리키도록 한다.
- temp = prev1.next

- `prev1.next = prev2.next`

3. `prev2.next`가 `x`를 가리키도록 한다.

- `prev2.next = temp`

Doubly Linked list

- 전제 : `x, y`의 위치는 주어져있다.
- `x`와 `y`는 이전 노드와 다음 노드를 알고 있다. 이를 이용하여 스왑한다.
- 알고리즘은 아래와 같다.

`temp`로 `x, y`의 기존 포인터 저장

- `temp_prev = x.prev`
- `temp_next = x.next`

`x`에 `y` 정보 넣기

- `x.prev = y.prev`
- `x.next = y.next`

`y`에 `x` 정보 넣기

- `y.prev = temp_prev`
- `y.next = temp_next`

주변 노드 정리

- `x.prev.next = x`
- `x.next.prev = x`
- `y.prev.next = y`
- `y.next.prev = y`

성능 비교

- Single Linked List는 `prev1, prev2`를 찾아야 하므로 리스트의 크기가 커질 수록 탐색 기간이 길어진다.
- 이러한 이유로 성능이 Doubly Linked List보다는 떨어진다.

Problem 10.

Given two nodes `x` and `y` that may belong to circularly linked lists (not necessarily the same one), describe an efficient algorithm to determine whether they belong to the same list.

```

current = x

while True:
    if current == y:
        return True
    current = current.next
    if current == x:
        return False

```

Problem 11.

In a doubly linked list with head and tail references, describe a non-recursive method to find the middle node by using only link traversal (no counter). If there is an even number of nodes, return the one slightly left of center.

- 양방향 연결 리스트에서 가장 중요한 것은 방향이 자유롭다는 점이다.
- 이를 이용해서 head와 tail에서 각각 출발해서 노드를 탐색하면 된다.
- 노드가 홀수가 되는 경우, head와 tail에서 출발하면 나중에 중간 노드에서 만나게 된다.
- 노드가 짝수가 되는 경우, head와 tail에서 출발하면 나중에 교차되는 순간이 있다.
- 이를 감안해서 아래와 같은 알고리즘을 짤다.

```

curr1 = head
curr2 = tail

while left != right and left.next != right:
    left = left.next
    right = right.prev

middleNode = left

```

Problem 12.

Describe an algorithm to reverse a singly linked list without using recursion and using only constant extra space.

- constant extra space 이므로 prev, curr, next을 사용할 수 있다.
- head 부터 tail 까지 진행하면서 reverse를 진행한다.
- prev, curr, next 3개 노드를 옮겨가면서 curr이 가리키는 노드의 next 방향을 바꿔준다.

```
prev = None  
curr = head
```

```
while curr is not None:
```

```
    next = curr.next # 다음 노드 저장  
    curr.next = prev # 링크 방향 반전  
    prev = curr # prev 한 칸 이동  
    curr = next # curr 한 칸 이동
```

```
head = prev # 마지막 prev가 새로운 head 이다.
```